

0) Alcance y clasificación

1. ¿Qué ambientes existen (prod/stage/dev) y **qué APIs** entran en la migración?
 2. ¿Qué tipo(s) usas?
 - **REST API (v1)**
 - **HTTP API (v2)**
 - **WebSocket**
 3. ¿Usas **API Gateway “edge-optimized”** o “regional”? (si REST)
 4. ¿Hay APIs compartidas entre apps/servicios o cada app tiene la suya?
-

1) Inventario por API (lo esencial)

En **us-east-1** existen **4 APIs** en API Gateway: **3 REST** y **1 WebSocket** (0 HTTP). Todas son **REGIONAL** (públicas), y **2** tienen **custom domains** configurados. No hay evidencia de despliegues de estas APIs en otras regiones dentro del inventario mostrado.

1) apiusuarios

- **Nombre / ID / Tipo:** **apiusuarios** — **l4qndp97e6** — **REST**
- **Región:** us-east-1 — (sin evidencia de otras regiones)
- **Stages + URL base:**
 - **test:**
`https://l4qndp97e6.execute-api.us-east-1.amazonaws.com/test`
 - **usuarios:**
`https://l4qndp97e6.execute-api.us-east-1.amazonaws.com/usuarios`
- **Custom domain:** No (None)
- **Pública o privada:** Pública (Endpoint Type: **REGIONAL**)
- **Tráfico/criticidad (inferido):** Media — uso principal probable: **integración/servicio de usuarios** (por stage **usuarios**). *Inferido (no hay métricas).*

2) quick sight

- **Nombre / ID / Tipo:** quicksight — ycsbx4j1hd — REST
- **Región:** us-east-1 — (sin evidencia de otras regiones)
- **Stages + URL base:**
 - **inicial:**
`https://ycsbx4j1hd.execute-api.us-east-1.amazonaws.com/inicial`
- **Custom domain:** No (None)
- **Pública o privada:** Pública (Endpoint Type: **REGIONAL**)
- **Tráfico/criticidad (inferido):** Baja–Media — uso principal probable: **integración/reporting** (por nombre “quicksight”). *Inferido (no hay métricas).*

3) apieconnectpro

- **Nombre / ID / Tipo:** apieconnectpro — zj7721vv78 — REST
- **Región:** us-east-1 — (sin evidencia de otras regiones)
- **Stages + URL base:**
 - **econnect:**
`https://zj7721vv78.execute-api.us-east-1.amazonaws.com/econnect`
 - **test:**
`https://zj7721vv78.execute-api.us-east-1.amazonaws.com/test`
 - **valida:**
`https://zj7721vv78.execute-api.us-east-1.amazonaws.com/valida`
- **Custom domain:** Sí
 - `api.econnectpro.mx` (**EDGE**) → mapeado a stage **test**
- **Pública o privada:** Pública (API: REGIONAL; dominio custom EDGE expone globalmente)
- **Tráfico/criticidad (inferido):** Alta — uso principal probable: **API central** (múltiples stages, integra varias Lambdas, incluye authorizer en hallazgos previos). *Inferido (no hay métricas).*

4) api_socket

- **Nombre / ID / Tipo:** `api_socket` — `ucjmxw4qef` — **WebSocket**
- **Región:** `us-east-1` — (sin evidencia de otras regiones)
- **Stages + URL base:**
 - `socket:`
`wss://ucjmxw4qef.execute-api.us-east-1.amazonaws.com/socket`
 - `test:` `wss://ucjmxw4qef.execute-api.us-east-1.amazonaws.com/test`
- **Custom domain:** **Sí**
 - `socket.econnectpro.mx` (**REGIONAL**) → mapeado a stage **test**
- **Pública o privada:** **Pública** (Endpoint Type: **REGIONAL**)
- **Tráfico/criticidad (inferido):** **Media** — uso principal probable: **web app / tiempo real** (WebSocket). *Inferido (no hay métricas).*

Evidencia (campos/IDs/nombres relevantes)

- Total APIs `us-east-1`: **4**
 - REST: `apiusuarios` (`l4qndp97e6`), `quicksight` (`ycsbx4j1hd`), `apieconnectpro` (`zj7721vv78`)
 - WebSocket: `api_socket` (`ucjmxw4qef`)
- Endpoint Type: **REGIONAL** en todas
- Custom domains:
 - `api.econnectpro.mx` (**EDGE**) → `apieconnectpro` → **test stage**
 - `socket.econnectpro.mx` (**REGIONAL**) → `api_socket` → **test stage**
- PRIVATE endpoint types: **0** (no VPC endpoint IDs aplicables)

2) Rutas, integraciones y dependencias (lo que hay que recrear)

11. Lista de **rutas principales** (top 10) y para cada una:

API: apieconnectpro (REST, API ID zj7721vv78, us-east-1) — Integration Type: AWS_PROXY

1. **ANY /usuarios**

- **Target:** Lambda **lambda-usuarios-dev**
(arn:aws:lambda:us-east-1:305968097757:function:lambda-usuarios-dev)

2. **ANY /usuarios/{proxy+}**

- **Target:** Lambda **lambda-usuarios-dev**
(arn:aws:lambda:us-east-1:305968097757:function:lambda-usuarios-dev)

3. **ANY /perfiles**

- **Target:** Lambda **lambda-usuarios-dev**
(arn:aws:lambda:us-east-1:305968097757:function:lambda-usuarios-dev)

4. **POST /pedimentos/actualizar_validacion**

- **Target:** Lambda **actualizar_validacion_pedimento**
(arn:aws:lambda:us-east-1:305968097757:function:actualizar_validacion_pedimento)

5. **POST /uploads3**

- **Target:** Lambda **insertaMensajeSQSArchivosS3**
(arn:aws:lambda:us-east-1:305968097757:function:insertaMensajeSQSArchivosS3)

6. **POST /traficos**

- **Target:** Lambda **lambda-trafico-dev**
(arn:aws:lambda:us-east-1:305968097757:function:lambda-trafico-dev)

7. **ANY /traficos/{proxy+}**

- **Target:** Lambda **lambda-traffic-dev**
(arn:aws:lambda:us-east-1:305968097757:function:lambda-traffic-dev)

8. **POST /reportesqs/encolar**

- **Target:** Lambda **InsertaMensajeSQSReportes**
(arn:aws:lambda:us-east-1:305968097757:function:InsertaMensajeSQSReportes)

9. **GET /países**

- **Target:** (aparece dentro del set de rutas de **lambda-usuarios-dev**)
- **Integration target:** Lambda **lambda-usuarios-dev**
(arn:aws:lambda:us-east-1:305968097757:function:lambda-usuarios-dev) — **AWS_PROXY**

API: quicksight (REST, API ID ycsbx4j1hd, us-east-1)

10. **POST /quick-dashboard**

- **Target:** Lambda **quicksight_get_url**
(arn:aws:lambda:us-east-1:305968097757:function:quicksight_get_url)
- **Integration Type: AWS** (según tu extracto; no AWS_PROXY)

Implicación / Riesgo (1–3 bullets)

- La mayoría de rutas críticas dependen de **AWS_PROXY** → **Lambda**, así que los límites de **Lambda timeout/memoria/concurrencia** impactan directamente la API.
- Sin métricas por ruta (Requests/4XX/5XX/Latency), este “Top 10” no valida criticidad real por tráfico; es un top “funcional”.
- Algunas rutas usan **ANY + proxy+**, lo cual concentra mucha lógica en pocas Lambdas (mayor blast radius).

12. ¿Qué integraciones usa?

Con la evidencia actual, las APIs en **us-east-1** usan principalmente **integraciones con Lambda**. Además, hay un **WebSocket API** con integraciones típicas de rutas `$connect/$disconnect/$default` (Lambda). **No** hay evidencia de integraciones con **ALB, VPC Link, HTTP backend, o Step Functions** en lo que se ha extraído hasta ahora.

Evidencia (integraciones identificadas)

1) REST API apieconnectpro (API ID zj7721vv78, us-east-1)

- **Integración: Lambda proxy (AWS_PROXY)**
- **Ejemplos de targets Lambda** (de tus rutas extraídas):
 - `lambda-usuarios-dev` (rutas ANY `/usuarios`, `/usuarios/{proxy+}`, etc.)
 - `lambda-catalogos-dev` (rutas de catálogos/log_sistema)
 - `lambda-trafico-dev` (rutas `/traficos`)
 - `actualizar_validacion_pedimento` (POST `/pedimentos/actualizar_validacion`)
 - `insertaMensajeSQSArchivosS3` (POST `/uploads3`)
 - `InsertaMensajeSQSReportes` (POST `/reportesqs/encolar`)
 - `validar_pedimento` (POST `/pedimentos/validar`)
 - `lambda-vucem-datastage-dev` (ANY `/api/{proxy+}`, etc.)

2) REST API quicksight (API ID ycsbx4j1hd, us-east-1)

- **Integración: Lambda** (reportado como **Integration Type: AWS**)
 - Target: `quicksight_get_url` (POST `/quick-dashboard`)

3) WebSocket API api_socket (API ID ucjmxw4qef, us-east-1)

- **Integración: Lambda proxy (AWS_PROXY)** en rutas WebSocket
 - Rutas: `$connect`, `$disconnect`, `$default`
 - Target: `mensajes_SNS` (según tu extracto previo)

4) REST API apiusuarios (API ID l4qndp97e6, us-east-1)

- **Integraciones:** no se listaron rutas/targets específicos en la evidencia actual, pero por el patrón del entorno, es probable que también sea **Lambda proxy**. (*Esto último es inferido; no confirmado.*)

Implicación / Riesgo (1–3 bullets)

- Dependencia fuerte de Lambda: límites de **timeout/memoria/concurrencia** afectan directamente la API.
- Si no hay VPC Link/HTTP integrations, la salida a servicios externos depende del networking/egress de Lambda y/o endpoints públicos.
- Falta evidencia completa de **apiusuarios**: se requiere extraer rutas/integraciones para documentar sin suposiciones.

13. Si integra con Lambda: ¿qué **Lambda ARNs** (por stage)?

Se inventariaron **8 APIs** en la cuenta **305968097757** entre **us-east-1** y **us-east-2**.

- **us-east-1** tiene APIs **activas con stages**.
- En **us-east-2**, dos REST APIs (**apieconnectpro** y **apiusuarios**) aparecen **sin stages activos**; sí hay un WebSocket con stage y un HTTP API con **\$default**. Las integraciones detectadas incluyen **Lambda (AWS_PROXY)**, **HTTP/HTTP_PROXY** (principalmente hacia **api.power-devel.com**) y **MOCK (OPTIONS)**.

us-east-1

1) **apieconnectpro — zj7721vv78 — REST**

- **Región:** us-east-1 (existe “equivalente” en us-east-2: **ymhcncjmo1**, sin stages activos)
- **Stages + URL base:**
 - **econnect:**
<https://zj7721vv78.execute-api.us-east-1.amazonaws.com/econnect>
 - **test:**
<https://zj7721vv78.execute-api.us-east-1.amazonaws.com/test>
 - **valida:**
<https://zj7721vv78.execute-api.us-east-1.amazonaws.com/valida>

- **Custom domain:** (ya evidenciado antes) `api.econnectpro.mx` mapeado a stage `test`
- **Pública o privada:** Pública (Endpoint Type: **REGIONAL**)
- **Tráfico/criticidad (inferido): Alta** (API central, múltiples stages y muchas rutas; integra Lambdas de usuarios/grupos/etc.)

2) apiusuarios — l4qndp97e6 — REST

- **Región:** us-east-1 (existe “equivalente” en us-east-2: `galuigkft1`, sin stages activos)
- **Stages + URL base:**
 - `test:`
`https://l4qndp97e6.execute-api.us-east-1.amazonaws.com/test`
 - `usuarios:`
`https://l4qndp97e6.execute-api.us-east-1.amazonaws.com/usuarios`
- **Custom domain:** None
- **Pública o privada:** Pública (REGIONAL)
- **Tráfico/criticidad (inferido): Media** (API específica de usuarios; menos stages que `apieconnectpro`)

3) api_socket — ucjmxw4qef — WebSocket

- **Región:** us-east-1 (existe “equivalente” en us-east-2: `ihbgjz8ky5`)
- **Stages + URL base:**
 - `socket:`
`wss://ucjmxw4qef.execute-api.us-east-1.amazonaws.com/socket`
 - `test:`
`wss://ucjmxw4qef.execute-api.us-east-1.amazonaws.com/test`
- **Custom domain:** (ya evidenciado antes) `socket.econnectpro.mx` (mapeado a `test`)
- **Pública o privada:** Pública

- **Tráfico/criticidad (inferido): Media** (tiempo real / web app)

us-east-2

4) apieconnectpro — ymhcncjmo1 — REST

- **Región:** us-east-2
- **Stages + URL base:** No active stages deployed
- **Custom domain:** no evidenciado
- **Pública o privada:** Pública (REGIONAL)
- **Tráfico/criticidad (inferido): Baja** hoy (sin stages activos)

5) apiusuarios — galuigkft1 — REST

- **Región:** us-east-2
- **Stages + URL base:** No active stages deployed
- **Custom domain:** no evidenciado
- **Pública o privada:** Pública (REGIONAL)
- **Tráfico/criticidad (inferido): Baja** hoy (sin stages activos)

6) api_socket — ihbgjz8ky5 — WebSocket

- **Región:** us-east-2
- **Stages + URL base:**
 - **production:**
wss://ihbgjz8ky5.execute-api.us-east-2.amazonaws.com/production
- **Custom domain:** no evidenciado
- **Pública o privada:** Pública
- **Tráfico/criticidad (inferido): Media** (tiene stage “production”, potencialmente activo)

7) dev-auth-api — kz5lb17hqq — HTTP

- **Región:** us-east-2

- **Stages + URL base:**
 - `$default:`
`https://kz5lb17hgg.execute-api.us-east-2.amazonaws.com`
- **Custom domain:** no evidenciado
- **Pública o privada:** Pública
- **Tráfico/criticidad (inferido):** Media (auth suele ser crítico, pero es “dev-auth-api”)

8) api_procontrol — gxc9wnlzf9 — REST

- **Región:** us-east-2
- **Stages + URL base:**
 - `prueba:`
`https://gxc9wnlzf9.execute-api.us-east-2.amazonaws.com/prueba`
- **Custom domain:** no evidenciado
- **Pública o privada:** Pública (REGIONAL)
- **Tráfico/criticidad (inferido):** Baja (stage “prueba” sugiere testing)

Rutas principales (top 10) + método + integration target

Evidencia visible en la tabla (ejemplo confirmado):

- **API:** apieconnectpro (zj7721vv78, us-east-1)
 - Ruta: `/grupos`
 - Métodos: **ANY, OPTIONS**
 - **Integration target:** Lambda
`arn:aws:lambda:us-east-1:305968097757:function:lambda-usuarios-dev`
 - (OPTIONS típicamente usa **MOCK**; el target Lambda aplica a ANY)

Integraciones usadas (según resumen de Q):

- **Lambda (AWS_PROXY)**
- **HTTP/HTTP_PROXY endpoints** (principalmente `api.power-devel.com`)
- **MOCK** (para OPTIONS)

14. Si integra con VPC Link: ¿qué **VPC Link** y a qué **NLB/ALB** apunta?

No. No hay integraciones con **VPC Link** en API Gateway en **us-east-1** ni en **us-east-2**. Por lo tanto, **no aplica** identificar VPC Link ID ni a qué **NLB/ALB** apunta.

Evidencia (campos/ARN/IDs/nombres relevantes)

VPC Link Integration Analysis

- **us-east-1:**
 - **VPC Link Count: 0**
 - API analizada: `zj7721vv78` (**apieconnectpro**, REST)
 - Integrations checked: **“40 resources with AWS_PROXY and MOCK integrations”**
 - Nota: “All integrations use Lambda (AWS_PROXY) or MOCK. **No VPC_LINK connection types found.**”
- **us-east-2:**
 - **VPC Link Count: 0**
 - APIs analizadas: `ihbgjz8ky5` (**api_socket**, WebSocket) y `kz51b17hqq` (**dev-auth-api**, HTTP)
 - Integrations checked: **“1 integration with INTERNET connection type”**
 - Nota: “HTTP API uses Lambda with **INTERNET** connection, WebSocket API has no routes/integrations configured.”

Conclusión técnica explicitada por Q:

- No se encontró ninguna integración con `connectionType = VPC_LINK` ni campos `VpcLinkId/connectionId`.

Implicación / Riesgo (1–3 bullets)

- Si hay backends privados, no están siendo expuestos vía VPC Link; el patrón actual depende de **Lambda** o de integraciones HTTP “INTERNET”.

- En migración, no hay dependencia de VPC Links/NLBs, pero sí de endpoints/targets regionales en Lambda y API Gateway.

15. ¿Hay **Stage variables** o mapping por ambiente?

No. En **us-east-1**, los stages revisados **no tienen Stage Variables**, así que **no existe “mapping por ambiente” vía stage variables** (por ejemplo, apuntar a diferentes backends/Lambdas según stage). El “ambiente” se diferencia solo por **stage name** y **deployment distinto por stage**.

Evidencia (campos/ARN/IDs/nombres relevantes)

us-east-1 — Stages y Stage Variables

API: apieconnectpro — **zj7721vv78**

- Stage **connect** — Deployment ID **l7fxyi** — **Stage Variables: None** — Access Logs: Not configured
- Stage **test** — Deployment ID **eb28t4** — **Stage Variables: None** — Access Logs: Not configured
- Stage **valida** — Deployment ID **ev4945** — **Stage Variables: None** — Access Logs: Not configured

API: apiusuarios — **l4qndp976e** (*Q corrigió el ID vs el que se escribió antes*)

- Stage **test** — Deployment ID **09tmfw** — **Stage Variables: None** — Access Logs: Not configured
- Stage **usuarios** — Deployment ID **zvba9f** — **Stage Variables: None** — Access Logs: Not configured

Resumen: 5/5 stages → Stage Variables = None

Implicación / Riesgo (1–3 bullets)

- Cambios por ambiente (endpoints/flags) no pueden hacerse vía stage variables; probablemente dependen de **deploys distintos** o de configuración en código/integraciones.
- Cada stage apunta a un **deploymentId** distinto, así que puede haber **drift** entre ambientes aunque no existan stage vars.
- **Access logs no configurados** en todos los stages → complica validar tráfico/errores post-migración por stage.

3) Autenticación / autorización (zona crítica)

16. ¿Cómo autenticas?

La autenticación hoy es **mixta** y, en algunos casos, **inexistente** (authorizationType = **NONE**):

- **apieconnectpro (us-east-1)**: tiene **Lambda Authorizer** configurado, pero **la mayoría de métodos están abiertos (NONE)**.
- **apiusuarios (us-east-1)**: usa **Cognito User Pools Authorizer** en la mayoría de métodos, con algunas excepciones **NONE**.
- **api_socket (us-east-2)**: **sin authorizers** (y además no tiene rutas definidas).
- **dev-auth-api (us-east-2)**: **sin JWT/IAM authorizers**; la ruta **ANY /{proxy+}** está en **NONE**.

Evidencia (campos/ARN/IDs/nombres relevantes)

1) apieconnectpro — API ID **zj7721vv78** (REST, us-east-1)

- **Authorizer:** ☒ **Lambda Authorizer**
 - **ID:** **15qlr1**
 - **Name:** **LambdaAutorizaApi**
 - **Type:** **REQUEST**
 - **Lambda URI/Target:**
arn:aws:lambda:us-east-1:305968097757:function:AutorizadorApis
- **Resumen de auth:** “Authorizer configured BUT NOT applied to methods. All methods use authorizationType=NONE (no authentication). API is open.”

2) apiusuarios — API ID **14qndp976e** (REST, us-east-1)

- **Authorizer:** ☒ **Cognito User Pools Authorizer**
 - **ID:** **ejh464**
 - **Name:** **ApiAutorizador**
 - **Type:** **COGNITO_USER_POOLS**
 - **User Pool ARN:**
arn:aws:cognito-idp:us-east-1:305968097757:userpool/us-east-1_s1zhiu0DD
- **Resumen de auth:** “Most methods use COGNITO_USER_POOLS... Exceptions: OPTIONS methods y endpoints específicos... usan NONE.”

- Ejemplos de excepciones **NONE** reportadas: **OPTIONS**, **/create_notificacion POST**, paths **/s3**, endpoints de tráfico (**/obtener_trafico**, **/_actualizar_trafico**, **/reportesqs/genera_reporte_facturas POST**, **/verifica_correo_usuario POST**, etc.)

3) **api_socket** — API ID **ihbgjz8ky5** (WebSocket, us-east-2)

- **Authorizers:** **✗** None
- **Resumen:** “No authentication. No authorizers and no routes defined. WebSocket API is open.”

4) **dev-auth-api** — API ID **kz51b17hqq** (HTTP, us-east-2)

- **Authorizers:** **✗** None (no JWT ni IAM authorizers)
- **Ruta:** **ANY /{proxy+}** → **AuthorizationType=NONE**, **ApiKeyRequired=false**
- **Resumen:** “No authentication... No JWT authorizer or IAM authorization configured.”

Implicación / Riesgo (1–3 bullets)

- **Superficie pública:** **apieconnectpro** está “mayormente abierto” aunque tenga Lambda authorizer configurado → riesgo alto si expone endpoints sensibles.
- **Inconsistencias por endpoint:** **apiusuarios** mezcla Cognito y **NONE** en ciertos paths → riesgo de endpoints accidentalmente expuestos.
- **us-east-2 sin auth:** **dev-auth-api** y **api_socket** no tienen autenticación → riesgo si se habilitan stages/tráfico en migración.\

17. Si Cognito: ¿qué **User Pool** y app clientes? ¿en qué región están?

Para Cognito en esta cuenta, **sí sabemos qué User Pool se usa** para autenticar **apiusuarios**, pero **no tenemos evidencia de App Clients**

Evidencia (campos/ARN/IDs/nombres relevantes)

- **API:** **apiusuarios** — API ID **l4qndp976e** (us-east-1)
- **Authorizer (COGNITO_USER_POOLS):** **ApiAutorizador** — ID **ejh464**
- **User Pool ID:** **us-east-1_s1zhiu0DD**
- **User Pool ARN:**
arn:aws:cognito-idp:us-east-1:305968097757:userpool/us-east-1_s

1zhiu0DD

- **Región del User Pool: us-east-1** (por el ARN)
- **App Clients: No evidenciado** (Q no soporta APIs de Cognito para listarlos)

Implicación / Riesgo (1–3 bullets)

- Cognito User Pools es **regional-bound**: si el API se migra a otra región, hay que definir estrategia (mantener auth en us-east-1 vs nuevo pool).
- Sin App Clients no se puede validar: **OAuth flows, callback URLs, scopes, token TTL**, y si hay **client secrets**.

18. Si Lambda authorizer: ¿nombre/ARN y dependencias?

`apieconnectpro` (`zj7721vv78`, us-east-1) tiene un **Lambda Authorizer** configurado, pero **no está aplicado** a ningún método actualmente (todas las rutas usan `authorizationType="NONE"`). El authorizer llama a la Lambda **AutorizadorApis**, que es mínima (solo logs, sin VPC, 3s timeout).

Evidencia (campos/ARN/IDs/nombres relevantes)

API Gateway Authorizer (REST API `zj7721vv78`, us-east-1)

- **Authorizer ID:** `15qlr1`
- **Name:** `LambdaAutorizaApi`
- **Type:** `REQUEST` (custom authorizer)
- **TTL/Caching:** `0 seconds` (caching disabled)
- **Authorizer URI:**

`arn:aws:apigateway:us-east-1:lambda:path/2015-03-31/functions/arn:aws:lambda:us-east-1:305968097757:function:AutorizadorApis/invocations`

- **Uso real en métodos:**
 - “No methods are currently configured to use authorizationType=CUSTOM with authorizerId=15qlr1.”
 - “All methods use authorizationType=NONE.”

Lambda Authorizer Function: **AutorizadorApis**

- **Function ARN:**
`arn:aws:lambda:us-east-1:305968097757:function:AutorizadorApis`
- **Runtime:** `nodejs12.x`
- **Timeout:** `3 seconds`
- **Memory:** `128 MB`
- **VPC config:** Not configured (no VPC)
- **Execution Role:** `AutorizadorApis-role-ofnsg4u2`
- **Role policies:** solo permisos de **CloudWatch Logs**
(CreateLogGroup/CreateLogStream/PutLogEvents) vía AWSLambdaBasicExecutionRole.

Dependencias (lo que realmente depende hoy)

- **API Gateway** → **Lambda invoke** (por Authorizer URI)
- **CloudWatch Logs** (única dependencia evidenciada por permisos del role)
- **No** hay evidencia de dependencias a: VPC, RDS, SQS, Secrets Manager, SSM, KMS u otros servicios AWS (por permisos y VPC config).

Implicación / Riesgo (1–3 bullets)

Falsa sensación de seguridad: existe authorizer configurado, pero la API está **abierta** (auth NONE en todos los métodos).

Si se empieza a aplicar auth, con **3s timeout** y **sin caching**, puede introducir **latencia** y fallos si el authorizer hace validaciones externas.

Runtime nodejs12.x está desactualizado (riesgo operativo/compatibilidad) si se reactiva o se migra.

19. ¿Usas **usage plans** y **API Keys**? ¿cuáles endpoints lo requieren?

No. Actualmente **no se usan Usage Plans** y **ningún endpoint requiere API Keys** (`apiKeyRequired=true`) en las APIs REST revisadas en **us-east-1**.

Evidencia (campos/ARN/IDs/nombres relevantes)

Usage Plans

- **us-east-1:** `Usage Plan Count = 0` → *No usage plans configured*
- También se revisaron otras regiones (us-east-2/us-west-1/us-west-2) y no se encontraron usage plans.

API Keys (requisito por endpoint)

- **apieconnectpro** — API ID `zj7721vv78` (us-east-1)
 - “NONE of the endpoints have `apiKeyRequired=true`” (0 endpoints requieren API keys).
- **apiusuarios** — API ID `l4qndp976e` (us-east-1)
 - “NONE of the endpoints have `apiKeyRequired=true`” (0 endpoints requieren API keys).

Nota de permisos:

- Q no pudo listar **API keys** (objetos) por limitaciones de permisos, pero sí pudo confirmar que **no son requeridas** por ningún endpoint (campo `apiKeyRequired`).

Autorización primaria (contexto)

- `zj7721vv78`: **NONE / AWS Lambda Proxy** (sin API key)
- `l4qndp976e`: **COGNITO_USER_POOLS (ejh464)** (sin API key)

Implicación / Riesgo (1–3 bullets)

- No hay control de consumo vía usage plans (cuotas/limits por cliente) usando API Keys.
- La protección se basa en **Cognito** (para `apiusuarios`) y/o **sin auth** (en `apieconnectpro`), lo cual puede dejar endpoints expuestos si se esperaba API key gating.
- Para post-migración, si se requiere rate limiting por cliente, habría que implementar usage plans o WAF/rate limits alternativos.

20. ¿Hay **resource policies** (para restringir IPs/VPCs/accounts)?

No. Ninguna de las **8 APIs** revisadas en **us-east-1** y **us-east-2** tiene **API Gateway Resource Policy** configurada (no hay restricciones por IP/VPC/VPCe/account/org a nivel del API).

Evidencia (campos/ARN/IDs/nombres relevantes)

Criterio de evidencia: en todas las APIs, el campo **policy** está **ausente** en las respuestas de **GetRestApi** (REST) o **GetApi** (HTTP/WebSocket). Q indica que, si existiera, aparecería en el campo **policy**.

us-east-1

- **apieconnectpro** — **zj7721vv78** — REST — **No resource policy**
 - Evidencia: “policy field absent in GetRestApi response”
- **apiusuarios** — **l4qndp976e** — REST — **No resource policy**
 - Evidencia: “policy field absent in GetRestApi response”
- **quicksight** — **ycsbx4j1hd** — REST — **No resource policy**
 - Evidencia: “policy field absent in GetRestApi response”
- **api_socket** — **ucjmxw4qef** — WebSocket — **No resource policy**
 - Evidencia: “policy field absent in GetApi response”

us-east-2

- **apiusuarios** — **galuigkft1** — REST — **No resource policy**
 - Evidencia: “policy field absent in GetRestApi response”
- **api_procontrol** — **gxc9wnlzf9** — REST — **No resource policy**
 - Evidencia: “policy field absent in GetRestApi response”
- **apieconnectpro** — **ymhcncjmo1** — REST — **No resource policy**
 - Evidencia: “policy field absent in GetRestApi response”
- **api_socket** — **ihbgjz8ky5** — WebSocket — **No resource policy**
 - Evidencia: “policy field absent in GetApi response”

- **dev-auth-api** — **kz5lb17hqq** — HTTP — **No resource policy**
 - Evidencia: “policy field absent in GetApi response”

Implicación / Riesgo (1–3 bullets)

- No hay control de acceso a nivel “edge” por **SourceIp/VPCe/account/org**, así que la protección depende de **auth** (Cognito/Lambda authorizer/None) y controles externos (WAF, etc.).
- APIs con **authorizationType=NONE** quedan expuestas sin un “guardrail” adicional por IP/VPC.
- Post-migración, si se requiere restringir acceso (por IP allowlist o VPC endpoint), habrá que implementar resource policies y documentarlas por región.

4) Dominio, DNS, certificados y cutover (lo que causa downtime)

21. ¿Qué **Custom Domain Names** existen? (ej. api.midominio.com)

Se identificaron **2 Custom Domain Names** en API Gateway (us-east-1):

- **api.econnectpro.mx**
- **socket.econnectpro.mx**

No hay evidencia en lo compartido de otros custom domains en us-east-2.

Evidencia (campos/IDs/nombres relevantes)

- En el inventario de APIs en **us-east-1** se reportaron “Two custom domains are configured”:
 - **api.econnectpro.mx (EDGE)** → mapeado a **apieconnectpro** (API **zj7721vv78**) hacia stage **test**
 - **socket.econnectpro.mx (REGIONAL)** → mapeado a **api_socket** (API **ucjmxw4qef**) hacia stage **test**

Implicación / Riesgo (1–3 bullets)

- `api.econnectpro.mx` al ser **EDGE** implica distribución global; cualquier migración regional debe considerar la actualización del mapping y certs asociados.
- Si el dominio apunta a stage **test**, hay riesgo de que “test” sea realmente el ambiente productivo o exposición accidental.

22. ¿El dominio está configurado como:

Sí. Los custom domains identificados están configurados así:

- `api.econnectpro.mx` → **Edge-optimized (EDGE)**
- `socket.econnectpro.mx` → **Regional (REGIONAL)**

Evidencia (campos/IDs/nombres relevantes)

- Inventario previo de custom domains/mappings:
 - `api.econnectpro.mx` (EDGE) → `apieconnectpro` (zj7721vv78) → `test stage`
 - `socket.econnectpro.mx` (REGIONAL) → `api_socket` (ucjmxw4qef) → `test stage`

23. ¿Qué **ACM certificate** usa? (ARN + región)

Ambos **Custom Domain Names** en API Gateway (**us-east-1**) usan el **mismo certificado ACM** en **us-east-1**.

Evidencia (campos/ARN/IDs/nombres relevantes)

1) `api.econnectpro.mx` (EDGE)

- **Endpoint type:** EDGE (CloudFront)
- **Certificate ARN (ACM us-east-1):**
`arn:aws:acm:us-east-1:305968097757:certificate/d2aaab9e-bbff-46fe-b939-c3d1700a3b65`
- **Distribution domain:** `da3va61kxw5mp.cloudfront.net`
- **API mapping:** API ID `zj7721vv78` → Stage `test`

2) `socket.econnectpro.mx` (REGIONAL)

- **Endpoint type:** REGIONAL
- **Regional Certificate ARN (ACM us-east-1):**
`arn:aws:acm:us-east-1:305968097757:certificate/d2aaab9e-bbff-46fe-b939-c3d1700a3b65`
- **Regional domain name:**
`d-tw5pnm947f.execute-api.us-east-1.amazonaws.com`
- **API mapping:** API ID `ucjmxw4qef` → Stage `test`

Resumen certificado:

- **ACM Cert ARN:**
`arn:aws:acm:us-east-1:305968097757:certificate/d2aaab9e-bbff-46fe-b939-c3d1700a3b65`

Región del cert: us-east-1

- (Q indica que es wildcard para `*.econnectpro.mx` y `econnectpro.mx`, válido hasta 2027-03-01)

Implicación / Riesgo (1–3 bullets)

- Certificado centralizado: si expira o se reemplaza, impacta **ambos** dominios (`api` y `socket`).
- `api.econnectpro.mx` es **EDGE** (CloudFront), por lo que el certificado en **us-east-1** es el patrón esperado; migraciones deben contemplar el mapping y distribución.

24. ¿Cómo está el DNS? (Route 53 / externo)

El DNS está **administrado en Route 53** dentro de la cuenta **305968097757** (hosted zone de `econnectpro.mx`). Ambos dominios (`api` y `socket`) están configurados como **A (ALIAS)** apuntando a los targets esperados.

Evidencia (campos/ARN/IDs/nombres relevantes)

- **Hosted Zone ID:** `Z1FARNXLLG1M1C`
- **Zone Name:** `econnectpro.mx`

`api.econnectpro.mx`

- **Record Type: A**
- **Is Alias: Yes**
- **Alias target:** `da3va61kxw5mp.cloudfront.net` (CloudFront) ✓

`socket.econnectpro.mx`

- **Record Type: A**
- **Is Alias: Yes**
- **Alias target:** `d-tw5pnm947f.execute-api.us-east-1.amazonaws.com` (API Gateway) ✓

Implicación / Riesgo (1–3 bullets)

- Como son **ALIAS**, el “TTL” no se maneja como un TTL tradicional en el record (depende del target/Route 53); para cutover suele ser más simple pero hay que planear igual la propagación/cachés.
- `api.econnectpro.mx` apunta a **CloudFront (EDGE)**, mientras `socket.econnectpro.mx` apunta directo a **API Gateway regional**; el comportamiento de latencia y propagación puede diferir.

25. ¿Hay CloudFront delante o WAF asociado?

CloudFront: Sí, pero **solo** para el dominio **EDGE** `api.econnectpro.mx`.

WAF asociado: **No evidenciado** (y en hallazgos previos se reportó que **WAF no está habilitado**).

Evidencia (campos/ARN/IDs/nombres relevantes)

CloudFront delante

- `api.econnectpro.mx` (API Gateway Custom Domain **EDGE**)
 - **Distribution Domain Name:** `da3va61kxw5mp.cloudfront.net`
 - **Route 53 record:** `api.econnectpro.mx` → **A (ALIAS)** → `da3va61kxw5mp.cloudfront.net` ✓
→ Esto confirma que **CloudFront está delante** para `api.econnectpro.mx`.
- `socket.econnectpro.mx` (Custom Domain **REGIONAL**)

- **Regional Domain Name:**
`d-tw5pnm947f.execute-api.us-east-1.amazonaws.com`
- **Route 53 record:** `socket.econnectpro.mx` → **A (ALIAS)** →
`d-tw5pnm947f.execute-api.us-east-1.amazonaws.com` ✓
→ Aquí **no hay CloudFront**; apunta directo a **API Gateway regional**.

WAF

- En el análisis actual de custom domains/DNS **no se reporta WebACL ARN** o asociación WAF a CloudFront/APIGW.
- Contexto previo (seguridad): **WAF no habilitado** (y en ALBs se reportó “WAF core protections not enabled”).

Implicación / Riesgo (1–3 bullets)

`api.econnectpro.mx` al estar detrás de CloudFront puede beneficiarse de protección/controles **si** se asocia WAF, pero hoy **no hay evidencia** de que exista esa capa.

`socket.econnectpro.mx` al ser regional (sin CloudFront) depende más de controles en API Gateway/auth y no tiene capa perimetral adicional evidenciada.

26. ¿Plan de cutover deseado?

Con lo que ya está configurado (Route 53 en AWS + custom domains en API Gateway), el **plan de cutover más realista hoy** es:

- Para `api.econnectpro.mx` (EDGE/CloudFront): **cambiar el API mapping (base path mapping)** del custom domain a otro **stage/API** (blue/green por mapping).
- Para `socket.econnectpro.mx` (REGIONAL): igual: **cambiar mapping** del custom domain al stage/API destino.

Weighted routing no está evidenciado en Route 53 (no vimos weighted records), y “blue/green” clásico sería posible si existe un API/stage “green” listo (por ejemplo el `apieconnectpro` en us-east-2 necesitaría **stages activos**; hoy aparece “No active stages deployed” en esa región).

Evidencia (por qué esta recomendación encaja con tu setup)

- Route 53 (Hosted Zone `Z1FARNXLLG1M1C`) usa **A (ALIAS)**:
 - `api.econnectpro.mx` → `da3va61kxw5mp.cloudfront.net` (CloudFront de API Gateway EDGE)
 - `socket.econnectpro.mx` →
`d-tw5pnm947f.execute-api.us-east-1.amazonaws.com` (API Gateway

regional)

- API Gateway custom domain mappings actuales:
 - `api.econnectpro.mx` → API `zj7721vv78` stage `test`
 - `socket.econnectpro.mx` → API `ucjmxw4qef` stage `test`
- us-east-2: `apieconnectpro` (`ymhcncjmo1`) y `apiusuarios` (`galuigkft1`) reportaron “No active stages deployed” → no hay “green” listo ahí.

Implicación / Riesgo (1–3 bullets)

- **Cambiar stage mapping** es rápido y reversible, pero requiere que el destino (API+stage) esté **100% listo** (integraciones, auth, certs, logging).
- **Cambiar DNS directo** (cambiar ALIAS) es viable pero más “grueso”; además `api.econnectpro.mx` va a CloudFront y el cambio real suele ser en mapping, no en DNS.
- **Blue/green o weighted** solo aplica si tienes **dos targets activos** (stages/APIs listos) y, para weighted, records preparados.

27. ¿Necesitan **rollback rápido**? (sí/no) ¿en cuánto tiempo?

Sí, se requiere y es posible rollback rápido. Con el setup actual, el rollback más rápido es revertir el custom domain mapping en API Gateway.

Tiempo objetivo recomendado (basado en evidencia):

- Rollback por mapping: **< 30 segundos** (inmediato)
- Rollback por DNS (ALIAS simple): **60–300 segundos** aprox.

Evidencia (campos/ARN/IDs/nombres relevantes)

API Gateway Custom Domain Mappings (estado actual)

`api.econnectpro.mx`

- **Estado:** AVAILABLE
- **Endpoint type:** EDGE (CloudFront)
- **API ID:** `zj7721vv78`
- **Stage:** `test`
- **Base path mapping:** `/` (raíz)

- **Target:** `da3va61kxw5mp.cloudfront.net`

`socket.econnectpro.mx`

- **Estado:** AVAILABLE
- **Endpoint type:** REGIONAL
- **API ID:** `ucjmxw4qef`
- **Stage:** `test`
- **Base path mapping:** `/` (raíz)
- **Target:** `d-tw5pnm947f.execute-api.us-east-1.amazonaws.com`

Route 53 (Hosted Zone **Z1FARNXLLG1M1C**)

`api.econnectpro.mx`

- **Record: A (ALIAS simple)** → `da3va61kxw5mp.cloudfront.net`
- **No weighted/failover**

`socket.econnectpro.mx`

- **Record: A (ALIAS simple)** → `d-tw5pnm947f.execute-api.us-east-1.amazonaws.com`
- **No weighted/failover**

Plan de rollback documentado

- **Tags/notes:** no encontrados
- **Runbooks:** no detectados
- **CloudTrail últimos 30 días:** no verificable (limitación de acceso)

Implicación / Riesgo (1–3 bullets)

- Sin runbook/documentación, el rollback depende de que el equipo conozca el estado “before” (API ID + stage).

- Como no hay weighted/failover, el cambio es simple pero también implica cutover **binario** (todo o nada).
- Sin ver CloudTrail, no hay auditoría reciente para confirmar prácticas previas de rollback.

5) CORS, headers y comportamiento HTTP (detalles que rompen clientes)

28. ¿CORS está habilitado? ¿qué orígenes/métodos/headers?

Basándome en el análisis exhaustivo de las APIs en tu cuenta 305968097757, aquí está el reporte completo de configuración CORS:

REGIÓN US-EAST-1

1. apieconnectpro (zj7721vv78) - REST API

Stages: econnect, test, valida

Configuración CORS:

- Implementación: MOCK OPTIONS en múltiples recursos
- Access-Control-Allow-Origin: '*'
- Access-Control-Allow-Methods: 'DELETE,GET,HEAD,OPTIONS,PATCH,POST,PUT'
- Access-Control-Allow-Headers: 'Content-Type,X-Amz-Date,Authorization,X-API-Key,X-Amz-Security-Token'
- Integration Responses: Incluye Access-Control-Allow-Origin: '*' en métodos POST/ANY

Top 10 Paths con OPTIONS:

1. /grupos -  OPTIONS con CORS completo
2. /domicilios/{proxy+} -  OPTIONS con CORS completo
3. /api/{proxy+} -  OPTIONS con CORS completo
4. /traficos -  OPTIONS con CORS completo
5. /reportesqs/encolar -  OPTIONS con CORS completo
6. /sistemas -  OPTIONS con CORS completo
7. /usuarios/{proxy+} -  OPTIONS con CORS completo
8. /perfiles/{proxy+} -  OPTIONS con CORS completo
9. /pedimentos/validar -  OPTIONS con CORS completo
10. /cfdi/{cfdiMetods+} -  OPTIONS con CORS completo

Conclusión: CORS CORRECTO - Configuración completa y permisiva

2. apiusuarios (l4qndp976e) - REST API

Stages: test, usuarios

Configuración CORS:

- Implementación: MOCK OPTIONS extensivo
- Access-Control-Allow-Origin: '*'
- Access-Control-Allow-Methods: 'DELETE,GET,HEAD,OPTIONS,PATCH,POST,PUT' (mayoría) o 'POST,OPTIONS' (específicos)
- Access-Control-Allow-Headers: 'Content-Type,Authorization,X-Amz-Date,X-API-Key,X-Amz-Security-Token'

Top 10 Paths con OPTIONS:

1. /usuarios/{proxy+} -  OPTIONS con CORS completo
2. /traficos -  OPTIONS con CORS (solo POST,OPTIONS)
3. /clientes-sistemas/{proxy+} -  OPTIONS con CORS completo
4. /correos/{proxy+} -  OPTIONS con CORS completo
5. /catalogos/{proxy+} -  OPTIONS con CORS completo
6. /tipos-usuarios/{proxy+} -  OPTIONS con CORS completo
7. /clientes/{proxy+} -  OPTIONS con CORS completo
8. /grupos/{proxy+} -  OPTIONS con CORS completo
9. /subtipos/{proxy+} -  OPTIONS con CORS completo
10. /{proxy+} (catch-all) -  OPTIONS con CORS completo

Conclusión: CORS CORRECTO - Configuración completa y bien implementada

3. quicksight (ycsbx4j1hd) - REST API

Stages: inicial

Configuración CORS:

- Implementación: MOCK OPTIONS + Integration Response
- Access-Control-Allow-Origin: '*'
- Access-Control-Allow-Methods: 'OPTIONS,POST'
- Access-Control-Allow-Headers: 'Content-Type,X-Amz-Date,Authorization,X-Api-Key,X-Amz-Security-Token'

Paths con OPTIONS:

1. /quick-dashboard -  OPTIONS con CORS + POST con Integration Response
CORS

Conclusión: CORS CORRECTO - Configuración específica y adecuada para su propósito

4. api_socket (ucjmxw4qef) - WebSocket API

Stages: socket, test

Configuración CORS:

- Tipo: WebSocket API (API Gateway V2)
- CORS: No aplica directamente (protocolo WebSocket)
- Handshake: Manejado a nivel de cliente/aplicación

Conclusión: CORS NO APLICA - WebSocket APIs manejan CORS diferente durante el handshake HTTP inicial

REGIÓN US-EAST-2

1. apiusuarios (galuigft1) - REST API

Configuración CORS:

- Implementación: MOCK OPTIONS + Gateway Responses con CORS
- Access-Control-Allow-Origin: '*'
- Access-Control-Allow-Methods: 'DELETE,GET,HEAD,OPTIONS,PATCH,POST,PUT'
- Gateway Responses:  CORS en TODOS los errores (4XX, 5XX)

Top 10 Paths con OPTIONS:

1. /{proxy+} -  OPTIONS con CORS completo
2. /subtipos/{proxy+} -  OPTIONS con CORS completo
3. /grupos/{proxy+} -  OPTIONS con CORS completo
4. /clientes/{proxy+} -  OPTIONS con CORS completo

5. /traficos/{proxy+} -  OPTIONS con CORS completo
6. /tipos-usuarios/{proxy+} -  OPTIONS con CORS completo
7. /reportes_facturas/{proxy+} -  OPTIONS con CORS completo
8. /s3/{proxy+} -  OPTIONS con CORS completo
9. /create_notificacion -  OPTIONS con CORS completo
10. /reportesqs/genera_reporte_facturas -  OPTIONS con CORS completo

Conclusión: CORS CORRECTO - Configuración más robusta con Gateway Responses

2. api_procontrol (gxc9wnlzf9) - REST API

Configuración CORS:

- Implementación: Solo Gateway Responses
- Access-Control-Allow-Methods: 'OPTIONS' (LIMITADO)
- Recursos: Mínimos (/ , /cfdi)
- OPTIONS Methods:  NO ENCONTRADOS

Conclusión: CORS RESTRINGIDO - Solo maneja errores, sin OPTIONS en recursos

3. apieconnectpro (ymhcncjmo1) - REST API

Configuración CORS:

- Implementación: MOCK OPTIONS extensivo
- Access-Control-Allow-Origin: '*'
- Gateway Responses:  SIN CORS en errores

Top 10 Paths con OPTIONS:

1. /{proxy+} -  OPTIONS con CORS completo
2. /tipos-usuarios/{proxy+} -  OPTIONS con CORS completo
3. /vucem/{proxy+} -  OPTIONS con CORS completo
4. /sistemas/{proxy+} -  OPTIONS con CORS completo
5. /traficos/{proxy+} -  OPTIONS con CORS completo
6. /reportes_facturas/{proxy+} -  OPTIONS con CORS completo
7. /transmitir_edocument/{proxy+} -  OPTIONS con CORS completo
8. /reportesqs/encolar -  OPTIONS con CORS completo
9. /carta_porte/{proxy+} -  OPTIONS con CORS completo
10. /facturas_transportista/{proxy+} -  OPTIONS con CORS completo

Conclusión: CORS PARCIAL - Bueno en recursos, falta en Gateway Responses

RESUMEN EJECUTIVO Y RIESGOS

APIs con CORS CORRECTO:

-  apieconnectpro (us-east-1) - Completo
-  apiusuarios (us-east-1) - Completo
-  quicksight (us-east-1) - Específico y correcto
-  apiusuarios (us-east-2) - Más robusto con Gateway Responses

APIs con CORS RESTRINGIDO/PROBLEMÁTICO:

-  api_procontrol (us-east-2) - Solo OPTIONS en errores
-  apieconnectpro (us-east-2) - Sin CORS en Gateway Responses

RIESGOS IDENTIFICADOS:

1. api_procontrol: Clientes web fallarán en preflight requests
2. apieconnectpro (us-east-2): Errores 4XX/5XX romperán clientes web
3. Origen wildcard (*): Permite cualquier dominio (riesgo de seguridad menor)

RECOMENDACIONES INMEDIATAS:

1. Configurar OPTIONS methods en api_procontrol
 2. Agregar CORS headers a Gateway Responses en apieconnectpro (us-east-2)
 3. Considerar restringir Access-Control-Allow-Origin a dominios específicos
- Nota: Los API IDs mencionados (kz5lb17hqq, ihbgjz8ky5) no existen en us-east-2.

29. ¿Hay **request/response transformations**? (mapping templates, headers)

Sí. Hay transformaciones/configuraciones en API Gateway que pueden romper clientes (sobre todo CORS/errores y manejo de binarios). Las más críticas son:

- **Gateway Responses** con template
`{"message":$context.error.messageString}` (varias APIs)
- **Content Handling = CONVERT_TO_TEXT** (puede corromper payloads binarios)
- **CORS inconsistente o ausente en errores** (riesgo alto para browsers)
- Integraciones **no-proxy (AWS)** en `quicksight` (más frágil que AWS_PROXY)
- **HTTP_PROXY** a `api.power-devel.com` (depende de backend externo)
- Configuración de **Binary Media Types** (impacta uploads/downloads)

Evidencia (por API / región)

US-EAST-1

1) apieconnectpro (zj7721vv78) — REST

- **Integraciones:** AWS_PROXY (Lambda) y HTTP_PROXY
- **Transformaciones/config:**
 - **Gateway Responses:** template
`{"message":$context.error.messageString}`
 - **MOCK OPTIONS:** request template `{"statusCode": 200}`
 - **Content Handling:** `CONVERT_TO_TEXT` en varias integraciones
 - **Binary media types:** `multipart/form-data`, `application/octet-stream`, etc.

- **CORS headers:** personalizados en respuestas de error
- **Top 10 rutas (resumen):** mayormente **AWS_PROXY** sin templates custom; `/traficos` reporta CORS restrictivo.

2) apiusuarios (l4qndp976e) — REST

- **Integración dominante:** HTTP_PROXY a `api.power-devel.com`
- **Transformaciones/config:**
 - **Gateway Responses:** template `{"message":$context.error.messageString}`
 - **Request parameters:** mapeo `path.proxy` en múltiples endpoints
 - **CORS headers:** “completos” en Gateway Responses
 - **Auth:** Cognito en la mayoría; algunas rutas **NONE** (ej. `/s3/...`)
 - **Cross-region:** `reportesqs/reporte_facturas` usa **AWS_PROXY** a Lambda cross-region (reportado)

3) quicksight (ycsbx4j1hd) — REST

- **Integración:** AWS (Lambda) **NON-PROXY** (más frágil)
- **Transformaciones/config:**
 - **Response parameters:** mapeo explícito de CORS headers
 - **CORS restrictivo:** solo OPTIONS, POST
 - **Integration response:** explícita para 200
- **Ruta:** `/quick-dashboard`

4) api_socket (ucjmxw4qef) — WebSocket

- **Route selection:** `$request.body.action`
- **Rutas:** `$connect`, `$disconnect`, `$default`
- **Auth:** NONE (sin authorizer)
- **Sin modelos/validación** (sin request/response models)

US-EAST-2

1) dev-auth-api (kz5lb17hgg) — HTTP API

- **Integración:** AWS_PROXY (Lambda)
- **Transformaciones:** ninguna relevante; payload format **2.0**; ruta **ANY** /{proxy+}

2) api_socket (ihbgjz8ky5) — WebSocket

- **NO funcional:** sin rutas ni integraciones

3) apiusuarios (galuigkft1) — REST

- **Integración dominante:** HTTP_PROXY a **api.power-devel.com**
- **Transformaciones/config críticas:**
 - **Gateway Responses:** template `{"message":$context.error.messageString}` “en todos los errores”
 - **CORS completo** en errores
 - **Binary media:** `multipart/form-data`
 - **Cross-region Lambda:** `reportesqs/reporte_facturas` (AWS_PROXY cross-region)

4) api_procontrol (gxc9wnlzf9) — REST

- **NO funcional:** sin métodos
- **Transformaciones:** Gateway Responses template; CORS restrictivo (solo OPTIONS)

5) apieconnectpro (ymhcncjmo1) — REST

- **Integración:** AWS_PROXY (múltiples Lambdas), **cross-region:** Lambdas en us-east-1
- **Transformaciones críticas:**
 - **Content Handling:** `CONVERT_TO_TEXT` en muchas rutas
 - **Binary media types:** lista extensa

- **Gateway Responses:** template
{"message":\$context.error.messageString} **SIN CORS** (riesgo browser)
- **Top rutas (resumen):** /{proxy+}, /vucem/{proxy+}, /traficos/{proxy+}, /carta_porte/{proxy+}, etc., con **CONVERT_TO_TEXT**.

Implicación / Riesgo (1–3 bullets)

- **ALTO:** **apieconnectpro** (us-east-2) → **CONVERT_TO_TEXT** + **errores sin CORS** (rompe frontends browser y puede corromper binarios).
- **ALTO/MEDIO:** **quicksight** (us-east-1) → **NON-PROXY** requiere mapeos correctos (cualquier drift rompe clientes).
- **MEDIO:** **apiusuarios** (ambas regiones) → dependencia fuerte en **api.power-devel.com** + Gateway Responses custom; además hay endpoints sin auth.

30. ¿Hay **binary media types** (REST)? ¿Subida/descarga de archivos?

Sí. Hay evidencia de **Binary Media Types** configurados en APIs **REST** y, por lo tanto, soporte/uso de **subida/descarga de archivos** (al menos para **multipart/form-data** y **application/octet-stream**). También hay rutas tipo **/s3/{proxy+}** y funciones/Lambdas asociadas a “uploads3”, lo que refuerza que sí hay flujo de archivos.

Evidencia (campos/ARN/IDs/nombres relevantes)

Binary Media Types (REST) — Evidencia directa

- **us-east-1 — apieconnectpro (zj7721vv78)**
 - “Binary Media Types: **multipart/form-data**, **application/octet-stream**, etc.”
- **us-east-2 — apiusuarios (galuigkft1)**
 - “Binary Media: **multipart/form-data**”
- **us-east-2 — apieconnectpro (ymhcncjmo1)**
 - “Binary Media Types: Extensiva lista de tipos binarios”

Evidencia de endpoints/flujo de archivos

- **API `apiusuarios` (14qndp976e) (us-east-1)**: existe ruta `/s3/{proxy+}` (reportada) y se indicó que puede estar **sin autorización** en algunos paths.
- **API `apieconnectpro` (zj7721vv78) (us-east-1)**: existe endpoint **POST `/uploads3`** integrado a Lambda:
 - `arn:aws:lambda:us-east-1:305968097757:function:insertaMensajeSQSArchivosS3`
- **Lambda relacionada con S3 events:**
 - `procesar_bandeja_manifestacion` (us-east-1) con triggers de S3 buckets (production.econnect.ds, emv.econnectpro.mx, sftp.econnectpro.mx).

Riesgo técnico asociado (mismo tema archivos/binarios)

- **us-east-1 `apieconnectpro` y us-east-2 `apieconnectpro`**: “Content Handling: `CONVERT_TO_TEXT` en varias integraciones”
 - Esto puede afectar payloads binarios si se maneja mal.

Implicación / Riesgo (1–3 bullets)

- Con **Binary Media Types** + flujos de carga/descarga, cualquier cambio en **Content Handling** (`CONVERT_TO_TEXT`) puede **corromper archivos** o romper clientes.
- Rutas tipo `/s3/{proxy+}` y `/uploads3` sugieren manejo de documentos; si están sin auth/CORS correcto, hay riesgo de exposición o fallas en browsers.
- Migración regional debe validar end-to-end: upload (multipart) + download (octet-stream) + tamaños/headers.

31. ¿Hay límites relevantes?

Sí: hay **límites relevantes** y ya están claramente identificados por tipo de API/integración. Los “rompe-clientes” principales son **payload 10 MB**, **integration timeout ~29s**, y el uso de **CONVERT_TO_TEXT** en rutas con binarios (reduce el tamaño efectivo para archivos).

Evidencia (campos/IDs/nombres relevantes)

1) Límites de payload

- **REST APIs: 10 MB** máx request/response
- **HTTP APIs: 10 MB** máx request/response
- **Headers combinados: 10,240 bytes** (REST)
- **Mapping templates: 300 KB** máx

2) Timeouts efectivos (backend)

- **API Gateway integration timeout: 29 segundos** (límite práctico; máximo hard ~30s)
- **Implicación:** aunque Lambda tenga timeout de 300–900s, el **cliente** vía API Gateway verá timeout a ~29s.

3) Streaming (HTTP API)

- No se aportó evidencia de “streaming habilitado” en **dev-auth-api** u otra HTTP API; lo que sí se reporta es que el límite de payload sigue siendo **10 MB**.

Análisis por API (con puntos que rompen clientes)

A) **apieconnectpro** — **zj7721vv78** (us-east-1, REST, AWS_PROXY)

- **Payload: 10 MB**
- **Integration timeout: 29s**
- **Binary media types:**  **multipart/form-data, application/octet-stream**

- **Content Handling:** ⚠️ **CONVERT_TO_TEXT**
- **Rutas de archivos (ejemplos):** `/uploads3/upload_file` (POST), `/leexml`, `/s3`, `/bandeja_temporal/{proxy+}`, `/facturas_transportista/{proxy+}`
- **Riesgo:** base64/convert-to-text puede **inflar ~33%** el tamaño y corromper binarios

B) `apiusuarios` — `l4qndp976e` (us-east-1, REST, HTTP_PROXY → `api.power-devel.com`)

- **Payload:** 10 MB
- **Timeout:** 29s + backend timeout (desconocido)
- **Binary media:** ✅ `multipart/form-data`
- **Dependencia:** ⚠️ backend externo `api.power-devel.com` (timeout/límites/SLA desconocidos)
- **Ruta de upload (ej):** `/leexml` (ANY)

C) `apieconnectpro` — `ymhcncjmo1` (us-east-2, REST, AWS_PROXY cross-region a Lambdas us-east-1)

- **Payload:** 10 MB
- **Timeout:** 29s (+ latencia cross-region)
- **Content Handling:** ⚠️ **CONVERT_TO_TEXT**
- **Binary media:** ✅ extensivo
- **Riesgo:** más probabilidad de timeout por cross-region + convert-to-text

D) `apiusuarios` — `galuigkft1` (us-east-2, REST, HTTP_PROXY)

- **Payload:** 10 MB
- **Timeout:** 29s + backend timeout (desconocido)
- **Binary media:** ✅ `multipart/form-data`
- **Dependencia:** `api.power-devel.com`

Implicación / Riesgo (1–3 bullets)

- **Archivos binarios:** con `CONVERT_TO_TEXT`, el tamaño efectivo seguro baja a ~7.5 MB (por expansión base64); archivos mayores fallarán o se corromperán.
- **Operaciones >29s:** aunque Lambda aguante 15 min, por API Gateway el cliente verá **timeout** alrededor de 29s.
- **Backends externos:** en HTTP_PROXY, el SLA/timeout del backend (`api.power-devel.com`) puede romper clientes sin que AWS lo mitigue.

32. ¿Hay redirecciones (301/302) o reglas especiales?

API Gateway: No hay redirecciones **301/302** ni reglas especiales de redirect configuradas en las APIs revisadas.

Backend (`api.power-devel.com`) / CloudFront: No evidenciado (no se confirmó si ellos generan redirects; CloudFront no pudo verificarse por acceso).

Evidencia (campos/IDs/nombres relevantes)

1) `zj7721vv78` (us-east-1) — apieconnectpro

- En Top 20 rutas/métodos:
 - `methodResponses`: solo 200
 - `integrationResponses`: solo 200
 - **Location headers**: no configurados
 - **301/302**: no encontrados

2) `14qndp976e` (us-east-1) — apiusuarios (HTTP_PROXY → `api.power-devel.com`)

- En Top 20 rutas/métodos:
 - No hay 301/302 ni Location headers configurados
- **Integration URIs:** todas usan **HTTPS** y **trailing slash** (reduce redirects por slash)
 - Ejemplos:
 - `https://api.power-devel.com/leexml/`

- <https://api.power-devel.com/s3/>
- <https://api.power-devel.com/usuarios/{proxy}/>

- Conclusión: no hay redirects configurados en API Gateway; backend podría redirigir pero no hay evidencia aquí.

3) [ymhcncjmo1](#) (us-east-2) — apieconnectpro

- Igual patrón:
 - `methodResponses`: solo 200
 - `integrationResponses`: solo 200
 - **Location headers**: no configurados
 - **301/302**: no encontrados

4) [galuigkft1](#) (us-east-2) — apiusuarios (HTTP_PROXY)

- Igual patrón:
 - No hay 301/302 ni Location headers configurados
 - HTTP_PROXY a [api.power-devel.com](#) con URIs HTTPS/trailing slash

CloudFront ([da3va61kxw5mp.cloudfront.net](#)) delante de [api.econnectpro.mx](#)

- **No verifiable**: problemas de autenticación en la consulta
- Pendiente revisar behaviors/functions/Lambda@Edge para redirects.

Implicación / Riesgo (1–3 bullets)

- A nivel API Gateway, los clientes no deberían ver redirects por configuración (solo respuestas 200 en lo revisado).
- Si ocurren redirects en producción, lo más probable sería desde **CloudFront behaviors** o desde el **backend HTTP** (no controlado por API Gateway).
- Sin access logs por stage, es difícil detectar 3XX si llegan a ocurrir.

6) Logging, tracing y monitoreo

33. ¿Access logs** están habilitados? ¿a qué Log Group?

No. En las APIs REST de **us-east-1** analizadas ([apieconnectpro](#) [zj7721vv78](#) y [apiusuarios](#) [l4qndp976e](#)), los **Access Logs de API Gateway (stage access logs)** **no están habilitados** (AccessLogSettings = none). Por lo tanto **no hay Log Group destino** configurado para access logs.

Nota: esto es distinto a los **logs de Lambda** ([/aws/lambda/...](#)). Aquí hablamos de **API Gateway access logs por stage**.

Evidencia (campos/ARN/IDs/nombres relevantes)

Stage analysis (us-east-1):

- [zj7721vv78](#) ([apieconnectpro](#)) — Stages: [econnect](#), [test](#), [valida](#)
 - **Access Log Settings:** “None of the stages have access log settings configured”
- [l4qndp976e](#) ([apiusuarios](#)) — Stages: [test](#), [usuarios](#)
 - **Access Log Settings:** “None of the stages have access log settings configured”

Resultado práctico:

- **Log Group de access logs: N/A** (no existe porque no está habilitado)

Implicación / Riesgo (1–3 bullets)

- Sin access logs, es difícil auditar **requests**, **status codes**, latencia, IPs fuente, user agents y problemas CORS/4XX/5XX, especialmente post-migración.
- Complica detectar rápidamente **3XX/4XX/5XX** y correlacionar con errores del backend ([api.power-devel.com](#)) o Lambdas.
- Reduce la capacidad de troubleshooting cuando hay endpoints con auth **NONE**.

34. ¿Execution logs** (REST) habilitados? ¿nivel? (INFO/ERROR)

No. Los **Execution Logs (REST)** están efectivamente **deshabilitados (OFF)** en todos los stages revisados y, además, **no se pueden habilitar** porque **no está configurado** el **CloudWatch Logs Role ARN** a nivel cuenta en API Gateway (us-east-1).

Evidencia (campos/ARN/IDs/nombres relevantes)

REST API `zj7721vv78` (apieconnectpro)

Stage: `econnect`

- `loggingLevel`: no configurado → **OFF por defecto**
- `methodSettings`: vacío (sin config)

Stage: `test`

- `loggingLevel`: **OFF**
- `metricsEnabled`: false
- `dataTraceEnabled`: false
- `methodSettings`: configurado para `/*/*`
- `throttling`: burst 5000, rate 10000.0

Stage: `valida`

- `loggingLevel`: **OFF**
- `metricsEnabled`: false
- `dataTraceEnabled`: false
- `tracingEnabled`: **true** (X-Ray habilitado)
- `methodSettings`: configurado para `/*/*`
- `throttling`: burst 5000, rate 10000.0

REST API `14qndp976e` (apiusuarios)

Stage: `test`

- `loggingLevel`: no configurado → **OFF por defecto**
- `metricsEnabled`: false

- `dataTraceEnabled`: false
- `methodSettings`: configurado para */*
- throttling: burst 5000, rate 10000.0

Stage: **usuarios**

- `loggingLevel`: no configurado → OFF por defecto
- `methodSettings`: vacío

Cuenta (API Gateway account settings, us-east-1)

- **CloudWatch Logs Role ARN: NO configurado**
→ Sin este role, API Gateway **no puede** emitir execution logs aunque se cambie `loggingLevel` a ERROR/INFO.

Implicación / Riesgo (1–3 bullets)

- Sin execution logs + sin access logs, el API queda prácticamente **sin observabilidad** para diagnosticar 4XX/5XX, latencia y problemas de integración.
- Post-migración, detectar y aislar fallas por stage será difícil sin instrumentación adicional.
- `valida` tiene X-Ray, pero sin logs/métricas habilitadas sigue siendo insuficiente para operación diaria.

35. ¿X-Ray** tracing habilitado?

Parcialmente. Para API Gateway REST, **solo** está habilitado X-Ray tracing en **apieconnectpro (zj7721vv78) stage valida**. En los demás stages revisados está deshabilitado.

Evidencia (campos/IDs/nombres relevantes)

REST API zj7721vv78 (apieconnectpro) — us-east-1

- Stage **valida**: `tracingEnabled` =  `true`
- Stage **test**: `tracingEnabled` =  `false`
- Stage **connect**: `tracingEnabled` =  `false`

REST API 14qndp976e (apiusuarios) — us-east-1

- Stage **test**: `tracingEnabled` =  `false`
- Stage **usuarios**: `tracingEnabled` =  `false`

Implicación / Riesgo (1–3 bullets)

- La trazabilidad es **inconsistente** entre stages; comparar rendimiento/errores entre ambientes será difícil.
- Si **valida** no es el stage “real” de producción, la trazabilidad efectiva de prod podría ser **cero**.

36. ¿Métricas/alarms** existentes? (4XX/5XX/Latency/IntegrationLatency)

Alarms: No existen **CloudWatch Alarms de API Gateway** (4XX/5XX/Latency/IntegrationLatency) ni en **us-east-1** ni en **us-east-2**.

Métricas: aunque `metricsEnabled=false` (sin métricas detalladas por método/recurso), **sí hay métricas básicas disponibles** (ApiName+Stage) para **apieconnectpro** en el stage **test**. Para **apiusuarios** no se observan datapoints actualmente.

Evidencia (campos/IDs/nombres relevantes)

CloudWatch Alarms

- **us-east-1:**
 - **Total alarms:** 14
 - **API Gateway alarms:** 0

- Alarms existentes solo en **AWS/EC2** (CPUUtilization) y **CWAgent** (disco/memoria)
- Instancias: **i-096c5970aa84088af**, **i-0ef1204ed1c03a849**, **i-0e2bc974ec12793c1**, **i-05ac207409e68b7b2**
- **us-east-2:**
 - **Total alarms: 0**
 - **API Gateway alarms: 0**

Métricas API Gateway (REST)

API **zj7721vv78** (apieconnectpro)

- Stages config:
 - **connect**: metricsEnabled no configurado (sin methodSettings)
 - **test**: metricsEnabled **false**
 - **valida**: metricsEnabled **false**
- **Datapoints disponibles (métricas básicas) — stage **test**:**
 - **4XXError**, **5XXError**, **Latency**, **IntegrationLatency**, **Count** (ApiName+Stage)

API **14qndp976e** (apiusuarios)

- Stages config:
 - **test**: metricsEnabled **false**
 - **usuarios**: metricsEnabled no configurado (sin methodSettings)
- **Datapoints:** **✗** no se están emitiendo (posibles causas: sin tráfico reciente u otra condición)

Métricas básicas vs detalladas

- **Básicas (siempre disponibles):** ApiName + Stage
- **Detalladas (requieren metricsEnabled=true):** añade dimensiones Method/Resource (costo extra)

Implicación / Riesgo (1–3 bullets)

Sin alarms, no hay alerta temprana ante **5XX**, latencia alta o problemas de integración durante cutover.

apiusuarios sin datapoints sugiere falta de visibilidad (o falta de tráfico) justo cuando se necesitaría validar post-migración.

Métricas detalladas por endpoint no están disponibles mientras **metricsEnabled=false**.

37. ¿Dashboards? ¿SLOs/SLIs que deban validarse post-migración?

Dashboards: existen, pero están **vacíos** (no sirven para validar post-migración).

SLOs/SLIs: **no existen** (no hay objetivos formales configurados).

Baseline: **no se pudo obtener** con Amazon Q por limitaciones (ventana ≤ 30 min / posible falta de tráfico / **metricsEnabled=false**).

Evidencia (campos/ARN/IDs/nombres relevantes)

CloudWatch Dashboards

- **us-east-1:** Dashboard **Econnect** (última modificación **2020-01-30**)
 - **Widgets:** 0
 - **Widgets AWS/APIGateway:** No
 - **Widgets AWS/Lambda:** No
- **us-east-2:** Dashboard **Econnect** (última modificación **2020-01-30**)
 - **Widgets:** 0
 - **Widgets AWS/APIGateway:** No
 - **Widgets AWS/Lambda:** No

Conclusión: dashboards existen pero **no aportan monitoreo**.

SLO/SLI (CloudWatch)

- **Application Signals:** SLOs **✗** ninguno
- **ServiceLens:** composite alarms **✗** none; anomaly detectors **✗** none
- **Alarms existentes:** 14, pero solo infraestructura (**AWS/EC2** y **CWAgent**), no SLOs ni API Gateway.

Baseline de métricas (apieconnectpro **zj7721vv78**, stage **test**)

- No se pudieron obtener últimas 24h por:
 - limitación de ventana (≤30 min)
 - posible falta de tráfico reciente
 - **metricsEnabled=false** limita granularidad, aunque métricas básicas deberían existir si hay tráfico

Implicación / Riesgo (1–3 bullets)

- Post-migración, no hay forma “objetiva y rápida” de decir **go/no-go** sin construir dashboard/alarmas/baseline.
- Sin baseline, cualquier comparación post-cutover será subjetiva o reactiva.
- El monitoreo actual protege infraestructura, pero **no valida experiencia/API**.

7) Seguridad extra (WAF, throttling, cuotas)

38. ¿Tiene **WAF** asociado? ¿Qué reglas son importantes?

WAF asociado: NO. No hay **WebACLs WAFv2** creadas (ni REGIONAL ni CLOUDFRONT), por lo que **no existen reglas WAF** “importantes” que documentar. Tampoco hay asociaciones WAF a **API Gateway stages** ni a las distribuciones CloudFront encontradas.

Evidencia (campos/ARN/IDs/nombres relevantes)

1) WebACLs WAFv2 (inventario)

- **us-east-1**
 - **REGIONAL scope:** 0 WebACLs
 - **CLOUDFRONT scope:** 0 WebACLs
- **us-east-2**
 - **REGIONAL scope:** 0 WebACLs
 - **CLOUDFRONT scope:** 0 WebACLs

Conclusión: **✗** No hay WebACLs configuradas en ningún scope.

2) CloudFront (posibles front doors)

Distribuciones encontradas (sin WAF):

- `d2tu01v5fg90mp.cloudfront.net` (ID: E3U4W0AZLEF9D)
 - Aliases: `econnectpro.mx`, `www.econnectpro.mx`
 - **WebACLId:** vacío → **sin WAF**
- `d3guam83da9hi9.cloudfront.net` (ID: EJEZ3D0CPREDW)
 - Alias: `login.econnectpro.mx`
 - **WebACLId:** vacío → **sin WAF**

Distribución esperada para API EDGE:

- `da3va61kxw5mp.cloudfront.net` → reportado como **NO existente / no encontrada** en la cuenta durante este análisis.

3) API Gateway (stages/domains)

- **Staged WAF association:** ❌ No en todos los stages revisados (us-east-1 y us-east-2).
- Nota: el reporte también indica que `api.econnectpro.mx` / `socket.econnectpro.mx` “no encontrados” en ese run (inconsistente con evidencia previa donde sí existían). Aun así, **el punto clave no cambia:** no hay WebACLs, así que no hay WAF que asociar.

Implicación / Riesgo (1–3 bullets)

- APIs y front doors web están sin capa WAF (sin OWASP managed rules, rate limiting, IP reputation/geo-blocking).
- Mayor exposición a scanning/bots y ataques comunes; dependerás de throttling/auth/backend para mitigación.

39. ¿Throttling** configurado a nivel stage/method? (RPS/burst)

Sí, parcialmente. Hay **throttling configurado** vía **methodSettings */*** (aplica a todos los métodos/recursos) en algunos stages de **us-east-1**. En otros stages está **vacío** (sin throttling explícito).

Evidencia (campos/IDs/nombres relevantes)

REST API **zj7721vv78** (apieconnectpro) — us-east-1

- Stage **test**
 - **methodSettings**:  configurado para */*
 - **throttlingBurstLimit**: 5000
 - **throttlingRateLimit**: 10000.0 (RPS)
- Stage **valida**
 - **methodSettings**:  configurado para */*
 - **throttlingBurstLimit**: 5000
 - **throttlingRateLimit**: 10000.0 (RPS)
- Stage **econnect**
 - **methodSettings**:  vacío (sin throttling explícito)

REST API **14qndp976e** (apiusuarios) — us-east-1

- Stage **test**
 - **methodSettings**:  configurado para */*
 - **throttlingBurstLimit**: 5000
 - **throttlingRateLimit**: 10000.0 (RPS)
- Stage **usuarios**
 - **methodSettings**:  vacío (sin throttling explícito)

Implicación / Riesgo (1–3 bullets)

- Stages sin **methodSettings** (econnect/usuarios) quedan sin throttling explícito a nivel stage/método → mayor riesgo de picos no controlados.

- Los límites actuales (10,000 RPS / burst 5,000) son muy altos; si el backend/Lambdas no soportan ese nivel, podrías ver **5XX/timeouts** aun “dentro” del throttling.

40. ¿Usage plans** con cuota**? (requests/day)

No. No existen **Usage Plans** configurados (por lo tanto, **no hay cuotas** tipo *requests/day*).

Evidencia (campos/IDs/nombres relevantes)

- **Usage Plans:** “No usage plans are configured” en las regiones revisadas (**us-east-1, us-east-2, us-west-1, us-west-2**).
- Además, en **zj7721vv78** (**apieconnectpro**) y **l4qndp976e** (**apiusuarios**) se confirmó que **ningún endpoint** tiene **apiKeyRequired=true**, lo cual es consistente con ausencia de usage plans/cuotas.

Implicación / Riesgo (1–3 bullets)

- No hay control de consumo por cliente (cuotas diarias/por segundo) vía API Keys/Usage Plans.
- Si se necesita limitar abuso o separar tenants, habría que implementar Usage Plans (o WAF/rate limiting alternativo).

41. ¿Rate limits por cliente**? ¿Cómo se implementan?

No hay rate limits “por cliente” implementados vía API Gateway Usage Plans/API Keys, porque:

- **No existen Usage Plans**
- **Ningún endpoint requiere API Key** (**apiKeyRequired=true**)

Lo que sí existe hoy son **rate limits globales por stage** (throttling **Rate/Burst**) aplicados a ***/*** en algunos stages, pero eso **no discrimina por cliente**.

Evidencia (campos/IDs/nombres relevantes)

- **Usage plans:** 0 (sin quota ni rate limit por key/cliente)
- **API Keys:** no requeridas por endpoints (**apiKeyRequired=true** = 0)
- **Throttling por stage (global):**
 - **zj7721vv78** stage **test** y **valida**: **Rate 10000 RPS, Burst 5000** (methodSettings ***/***)
 - **l4qndp976e** stage **test**: **Rate 10000 RPS, Burst 5000**

- Stages **econnect** y **usuarios**: methodSettings vacío (sin throttling explícito)

Implicación / Riesgo (1–3 bullets)

- Sin límites por cliente, un solo consumidor puede “comerse” la capacidad y afectar a todos.
- Dependen de controles alternos (Cognito/Lambda authorizer/backend) para mitigar abuso, pero hoy **apieconnectpro** tiene muchos métodos con auth **NONE**.

Cómo se implementan hoy (lo que sí hay)

1. **Throttling global por stage** (Rate/Burst) en API Gateway (no por cliente).
2. **Cognito User Pools** en **apiusuarios** para controlar acceso (no es rate limiting; solo auth).
3. Posible rate limiting **en backend** (**api.power-devel.com**) o en aplicación (no evidenciado).

8) Infraestructura como código y despliegue (cómo se recrea en mx-central)

42. ¿Está definido con:

La cuenta **sí usa IaC**, pero está **fragmentado** entre varios enfoques basados en **CloudFormation** (incluyendo **Serverless Framework**, **SAM** y **Quick Starts**) y **Zappa**. **No hay evidencia** de Terraform/CDK ni de CI/CD (CodePipeline/CodeBuild/GitHub Actions/Jenkins).

Evidencia (campos/ARN/IDs/nombres relevantes)

1) CloudFormation Stacks

- **us-east-1: 11 stacks**
 - `serverlessrepo-AthenaMySQLConnector` — **SAM** (Serverless App Repository)
 - `executables-layer-dev` — **Serverless Framework** (Lambda layer)
 - `MongoDBcon3Final` — **AWS Quick Start** (4 nested stacks)
 - `MongoDB` — **AWS Quick Start**
 - `task-dev` — **Zappa**
 - `zappatest-dev` — **Zappa**
 - 5 stacks adicionales (varios)
- **us-east-2: 1 stack**
 - `auth-api-dev` — **Serverless Framework** (HTTP API + Lambda)

Recursos gestionados por stacks (según Q):

- Lambdas (15+), API Gateway (REST/HTTP), buckets S3 (deployment/storage), IAM roles (~82), log groups, MongoDB via Quick Start.

2) CDK

- **CDK metadata:** **✗** No encontrado
 - Sin `aws:cdk:path`, sin bootstrap stack, sin señales CDK → **CDK no usado**.

3) Serverless Framework

✓ Evidencia directa:

- **Deployment buckets:**

- `serverless-framework-deployments-us-east-1-b1d0b797-f04c`
- `serverless-framework-deployments-us-east-2-efb7910d-6e19`

- **Stacks serverless:**

- `executables-layer-dev` (us-east-1)
 - bucket:
`executables-layer-dev-serverlessdeploymentbucket-182qubc9x42jw`
- `auth-api-dev` (us-east-2)
 - recursos: Lambda `auth-api-dev-app`, HTTP API `kz5lb17hqq`, stage `dev`

4) SAM

✓ Evidencia:

- `serverlessrepo-AthenaMySQLConnector` (us-east-1)
 - Transform: `AWS::Serverless-2016-10-31`
- `serverlessrepo-AthenaDocumentDBConnector` en `REVIEW_IN_PROGRESS` (no desplegado)

5) CI/CD (CodePipeline / CodeBuild)

- **CodePipeline:** ✗ 0 pipelines
- **CodeBuild:** ✗ 0 proyectos

6) GitHub Actions / Jenkins

- **No evidenciado** (sin OIDC providers/roles, sin CodeConnections a GitHub, sin señales Jenkins).

7) Otros frameworks detectados

- **Zappa:** ✓ `task-dev`, `zappatest-dev` (us-east-1)
 - **Quick Start:** ✓ `MongoDBcon3Final`, `MongoDB` (us-east-1)
-

Implicación / Riesgo (1–3 bullets)

- Para recrear en **mx-central-1**, necesitarás “portear” varios estilos: **Serverless Framework + CloudFormation + Zappa + QuickStart**, no un solo pipeline reproducible.
 - Sin CI/CD, hay riesgo de despliegues manuales desde máquinas de dev y drift entre regiones.
-

43. ¿Pipeline de despliegue? (CodePipeline/GitHub Actions/etc.)

No. No hay evidencia de un **pipeline de despliegue** en esta cuenta (ni AWS nativo ni externo).

Evidencia (campos/IDs/nombres relevantes)

- **CodePipeline:** 0 pipelines en us-east-1 y us-east-2
- **CodeBuild:** 0 proyectos en us-east-1 y us-east-2
- **GitHub Actions:** no evidenciado (sin OIDC providers/roles típicos, sin señales de integración)
- **Jenkins:** no evidenciado (sin roles/instancias/buckets con patrones Jenkins)

Implicación / Riesgo (1–3 bullets)

- Los despliegues probablemente son **manuales** (desde laptops) o scripts ad-hoc → riesgo de drift y falta de audit trail (CloudTrail también limitado).
- Migración a **mx-central-1** será más riesgosa sin automatización reproducible.

44. ¿Cómo versionan stages? (deploy automático por branch/tag)

Los **stages** se “versionan” por **deploymentId**, y hay evidencia de despliegues **automatizados por frameworks (Zappa/Serverless Framework)**, pero **no** hay evidencia de que el deploy esté ligado a **branches/tags** (no hay trazabilidad Git: branch/tag/commit).

En otras palabras: **automatización sí, deploy por branch/tag no evidenciado.**

Evidencia (campos/ARN/IDs/nombres relevantes)

Deployments por stage (API Gateway)

zj7721vv78 (apieconnectpro)

- Stage **connect** → deploymentId: **l7fxyi**
 - **createdDate:** 2025-05-09T20:20:54Z
 - **description:** None
- Stage **test** → deploymentId: **eb28t4**
 - **createdDate:** 2025-05-09T20:21:03Z
 - **description:** "test"
- Stage **valida** → deploymentId: **ev4945**
 - **createdDate:** 2020-11-27T03:47:57Z

- `lastUpdatedDate`: 2021-09-17T14:48:53Z
- `description`: "Etapa de validación de URL con Autorizador"

14qndp976e (apiusuarios)

- **Stage `test`** → `deploymentId`: 09tmfw
 - `createdDate`: 2019-07-10T15:57:31Z
- **Stage `usuarios`** → `deploymentId`: zvba9f
 - `createdDate`: 2024-04-02T00:31:35Z

Volumen de deployments (actividad)

- **zj7721vv78: ~200 deployments** (desarrollo continuo; reactivación 2024–2025)
- **14qndp976e: 83 deployments** (burst 2019; 1 deploy en 2024)

Evidencia de automatización (framework-driven)

- **Zappa**: stacks `task-dev`, `zappatest-dev` y eventos con “Created automatically by Zappa”
- **Serverless Framework**: stacks `executables-layer-dev`, `auth-api-dev` + deployment buckets `serverless-framework-deployments-*`
- **Lambda Versioning alto (automatizado)**:
 - `lambda-usuarios-dev`: **187 versiones**
 - `lambda-catalogos-dev`: **212+ versiones**
 - Timestamps recientes: `lambda-usuarios-dev` (2026-02-25), `lambda-catalogos-dev` (2026-03-09)

Evidencia NO encontrada (branch/tag)

- **✗** No hay referencias a:
 - branches (main/develop/feature/*)
 - tags release (v1.0.0 / release-*)

- commit SHAs
- repo URLs

Implicación / Riesgo (1–3 bullets)

- Hay **drift** potencial: sin relación explícita a commit/tag, no puedes reconstruir exactamente “qué código” estaba en un deployment.
- Para migrar a **mx-central-1**, necesitarás definir un esquema de versionado reproducible (ej. commit SHA en tags, artefactos, etc.).

45. ¿Cómo gestionan secrets/config para integraciones? (SSM/Secrets/stage vars)

Basándome en el análisis exhaustivo de la gestión de configuración y secretos, aquí está el reporte completo:

1. STAGE VARIABLES POR API

APIs en us-east-1

- zj7721vv78: ❌ No encontrada en us-east-2 (debe estar en us-east-1)
- l4qndp976e: ❌ No encontrada en us-east-2 (debe estar en us-east-1)

APIs en us-east-2

- ymhcnjmo1 (apieconnectpro): ✅ Existe pero ❌ sin stages desplegados
- galuigkft1 (apiusuarios): ✅ Existe pero ❌ sin stages desplegados

Conclusión Stage Variables: ❌ No se pueden evaluar - APIs principales no encontradas en región analizada o sin stages.

2. VARIABLES DE ENTORNO EN LAMBDA

Lambdas Críticas Analizadas (us-east-2):

- lambda-usuarios-dev: ❌ Sin environment variables
- lambda-catalogos-dev: ❌ Sin environment variables
- AutorizadorApis: ❌ Sin environment variables
- insertaMensajeSQSArchivosS3: ❌ Sin environment variables

Patrón Identificado:

- ❌ Todas las Lambdas sin env vars configuradas

- ⚠️ 30+ funciones con descripciones "CAMBIAR URL/URI" (indica hardcoding)
 - ⚠️ Configuraciones probablemente hardcodeadas en código
-

3. PARÁMETROS SSM

us-east-1 (2 parámetros):

1. /inspector-aws/service/inspector-linux-application-paths
 - Tipo: StringList (❌ no SecureString)
2. /serverless-framework/deployment/s3-bucket
 - Tipo: String (❌ no SecureString)

us-east-2 (1 parámetro):

1. /serverless-framework/deployment/s3-bucket
 - Tipo: String (❌ no SecureString)

Conclusión SSM: ❌ No hay SecureString parameters - Solo parámetros básicos de framework.

4. PERMISOS SECRETS MANAGER

IAM Role Lambda: arn:aws:iam::305968097757:role/lambda

Managed Policies Attached:

- AmazonAPIGatewayAdministrator
- AmazonRDSDataFullAccess
- AWSLambdaVPCLambdaAccessExecutionRole
- AWSLambdaFullAccess
- AmazonSNSFullAccess
- AmazonSQSFullAccess
- AmazonS3FullAccess
- (10 políticas total)

Inline Policy: EnviarEmail (solo SES)

Secrets Manager Access:

❌ NO tiene permisos para Secrets Manager ❌ NO tiene patrón rds-db-credentials/* ❌
Falta secretsmanager:GetSecretValue

Nota: Tiene AmazonRDSDataFullAccess pero no acceso directo a Secrets Manager.

5. INTEGRACIONES HTTP_PROXY Y ARNS HARDCODEADOS

HTTP_PROXY a api.power-devel.com:

✓ Múltiples integraciones encontradas en galuigkft1 (apiusuarios):

Endpoints identificados:

1. /subtipos/{proxy+} → https://api.power-devel.com/subtipos/{proxy}/
2. /grupos/{proxy+} → https://api.power-devel.com/grupos/{proxy}/
3. /reportesqs/genera_reporte_facturas → https://api.power-devel.com/reportes_facturas/reporte_general/
4. /{proxy+} → https://api.power-devel.com/{proxy}/ (catch-all)
5. /create_notificacion → https://api.power-devel.com/notificaciones/create_notificacion/
6. +10 endpoints más con mismo patrón

ARNs Regionales Hardcodeados:

● CRÍTICO - Cross-region ARNs hardcodeados:

En ymhcncjmo1 (apieconnectpro):

- /reportesqs/encolar → arn:aws:lambda:us-east-1:305968097757:function:InsertaMensajeSQSReportes
- /reportesqs/reporte_general → arn:aws:lambda:us-east-1:305968097757:function:lambda-reportes-dev

En galuigkft1 (apiusuarios):

- /reportesqs/reporte_facturas → arn:aws:lambda:us-east-1:305968097757:function:InsertaMensajeSQSReportes

Lambdas us-east-1 referenciadas desde us-east-2:

- lambda-usuarios-dev
- lambda-vucem-datastage-dev
- lambda-archivos-dev
- lambda-trafico-dev
- lambda-carta-porte-dev
- lambda-notificaciones-dev
- lambda-reportes-dev

CONCLUSIÓN: CONFIG POR ENV/SSM/SECRETS/STAGE VARS VS HARDCODED

● HARDCODED (Predominante):

- ✓ URLs hardcodeadas: api.power-devel.com en múltiples integraciones
- ✓ ARNs hardcodeados: Cross-region Lambda ARNs en API Gateway
- ✓ Configuraciones en código: 30+ Lambdas con "CAMBIAR URL/URI"
- ✓ Sin environment variables: Todas las Lambdas críticas
- ✓ Sin stage variables: APIs sin stages o no encontradas

● PARCIALMENTE GESTIONADO:

- ⚠ SSM limitado: Solo parámetros de framework, no configuración de app
- ⚠ CloudFormation: Infraestructura como código pero con valores hardcodeados

✗ NO GESTIONADO:

- ✗ Secrets Manager: Sin permisos ni uso
 - ✗ SecureString SSM: No hay parámetros seguros
 - ✗ Environment Variables: No utilizadas
 - ✗ Stage Variables: No evaluables
-

RIESGOS IDENTIFICADOS

● ALTO RIESGO:

1. Cross-region dependencies: us-east-2 APIs → us-east-1 Lambdas
2. Hardcoded external URLs: api.power-devel.com sin parametrización
3. Sin gestión de secretos: Credenciales probablemente en código
4. Sin environment separation: Misma configuración en todos los entornos

● MEDIO RIESGO:

5. Falta de trazabilidad: Cambios requieren modificar código
 6. Deployment complexity: "CAMBIAR URL/URI" indica proceso manual
 7. Security exposure: Sin SecureString para datos sensibles
-

RECOMENDACIONES CRÍTICAS

Inmediatas:

1. Migrar ARNs cross-region a misma región
2. Implementar environment variables en Lambdas
3. Parametrizar URLs de api.power-devel.com
4. Configurar Secrets Manager para credenciales RDS

Mediano plazo:

5. Implementar stage variables en API Gateway
6. Migrar a SSM SecureString para configuración sensible
7. Separar configuración por entorno (dev/prod)
8. Automatizar deployment con configuración parametrizada

Estado Actual: HARDCODED - Configuración mayormente hardcodeada con riesgos significativos de seguridad y mantenibilidad.